

Introduction to Intensive Programming in C++

Lecture 7: (some) Graph algorithms II

Elias TSIGARIDAS

École Polytechnique

Contents

MST

Boruvka

Prim

Kruskal

Shortest Path(s)

SSSP for unweighted graphs

SSSP for DAGs

Dijkstra

Bellman-Ford

All Pairs Shortest Paths

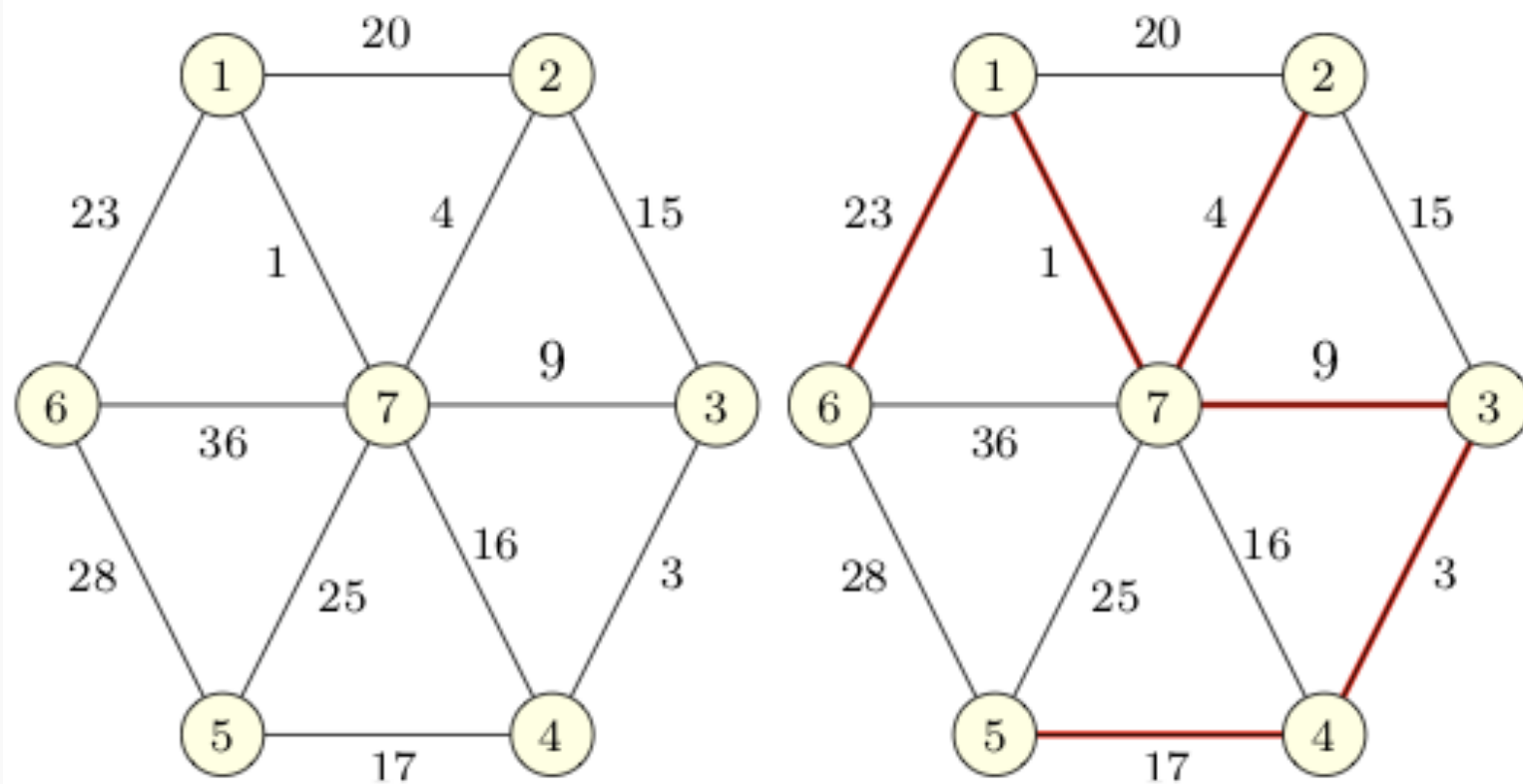
MST

Input

Connected graph $G = (V, E)$ with edge costs.

Output

Find $T \subseteq G$ such that (V, T) is connected and the total cost of all edges in T is the smallest possible.

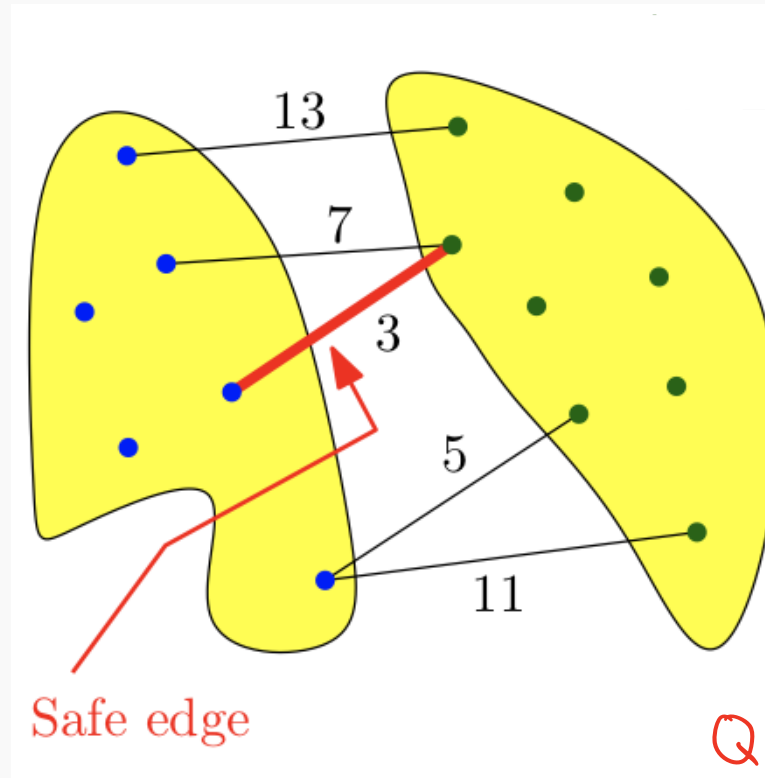


Unified Universal algorithm and safe edges

[Prim]: The MST contains every safe edge

$F \equiv \text{subgraph of } G = (V, E)$

- Initially F contains the n vertices (ie n forests)
- At each step the ALG adds SAFE edges to F



NOTE:

F is connected only at the end

Q: How do we pick the safe edges?

Safe edge

Minimum weight edge with **exactly** one endpoint in some component.

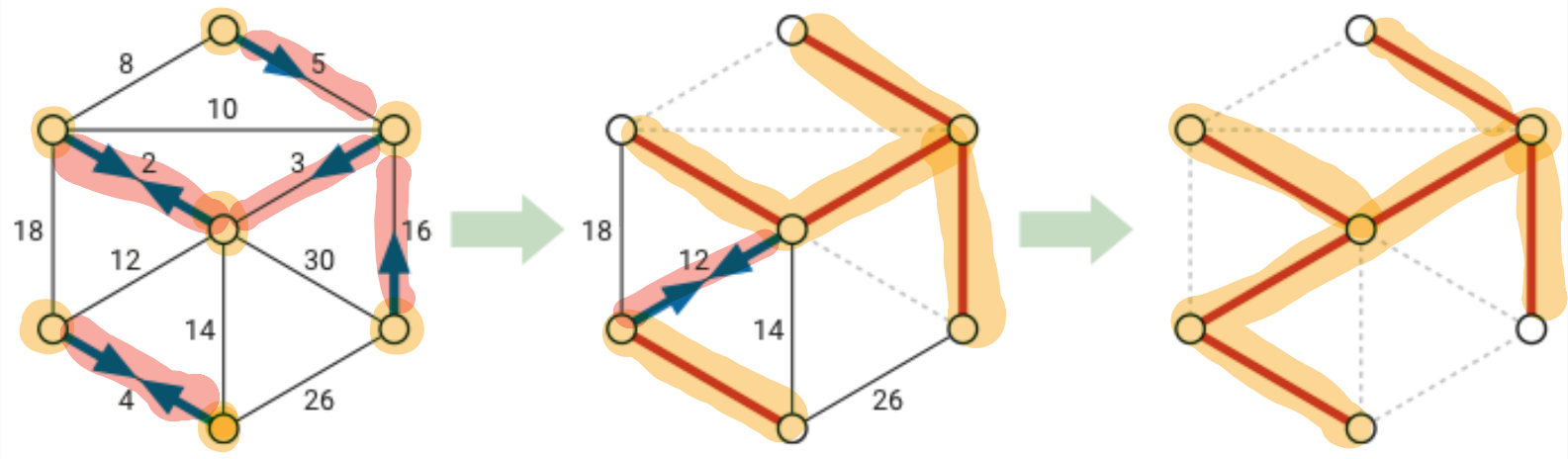
Boruvka 1926

Construct an electrical network
connecting various cities with
the least amount of wire
for the West Moravian Power Company

at each step we have
many forests

BORŮVKA: Add **ALL** the safe edges and recurse.

F



Boruvka's alg (pseudocode)

Complexity

$$O(E \lg V)$$

each iteration

reduces the components by
at least a factor of 2
so #iterations = $O(\lg V)$

BORUVKA(V, E):

$F = (V, \emptyset)$

$count \leftarrow \text{COUNTANDLABEL}(F)$

while $count > 1$

$\text{ADDALLSAFEEDGES}(E, F, count)$

$count \leftarrow \text{COUNTANDLABEL}(F)$

return F

Count the number of
connected components of F
and give a label to each

ADDALLSAFEEDGES(E, F, count):

for $i \leftarrow 1$ to $count$

$safe[i] \leftarrow \text{NULL}$

 for each edge $uv \in E$

 if $comp(u) \neq comp(v)$

 if $safe[comp(u)] = \text{NULL}$ or $w(uv) < w(safe[comp(u)])$

$safe[comp(u)] \leftarrow uv$

 if $safe[comp(v)] = \text{NULL}$ or $w(uv) < w(safe[comp(v)])$

$safe[comp(v)] \leftarrow uv$

for $i \leftarrow 1$ to $count$

 add $safe[i]$ to F

the safe
edge of each
component

each iteration
costs $O(E)$

Complexity $O(V+E)$

Easy parallelizable (consider each component)

Boruvka's alg (pseudocode)

Complexity

$$O(E \lg V)$$

each iteration

reduces the components by
at least a factor of 2
so #iterations = $O(\lg V)$

BORUVKA(V, E):

$F = (V, \emptyset)$

$count \leftarrow \text{COUNTANDLABEL}(F)$

while $count > 1$

$\text{ADDALLSAFEEDGES}(E, F, count)$

$count \leftarrow \text{COUNTANDLABEL}(F)$

return F

Count the number of
connected components of F
and give a label to each

How do we do this?

ADDALLSAFEEDGES(E, F, count):

for $i \leftarrow 1$ to $count$

$safe[i] \leftarrow \text{NULL}$

 for each edge $uv \in E$

 if $comp(u) \neq comp(v)$

 if $safe[comp(u)] = \text{NULL}$ or $w(uv) < w(safe[comp(u)])$

$safe[comp(u)] \leftarrow uv$

 if $safe[comp(v)] = \text{NULL}$ or $w(uv) < w(safe[comp(v)])$

$safe[comp(v)] \leftarrow uv$

for $i \leftarrow 1$ to $count$

 add $safe[i]$ to F

the safe
edge of each
component

How do we do this?

each iteration
costs $O(E)$

Complexity $O(V+E)$

Easy parallelizable (consider each component)

Prim-Jarník algorithm

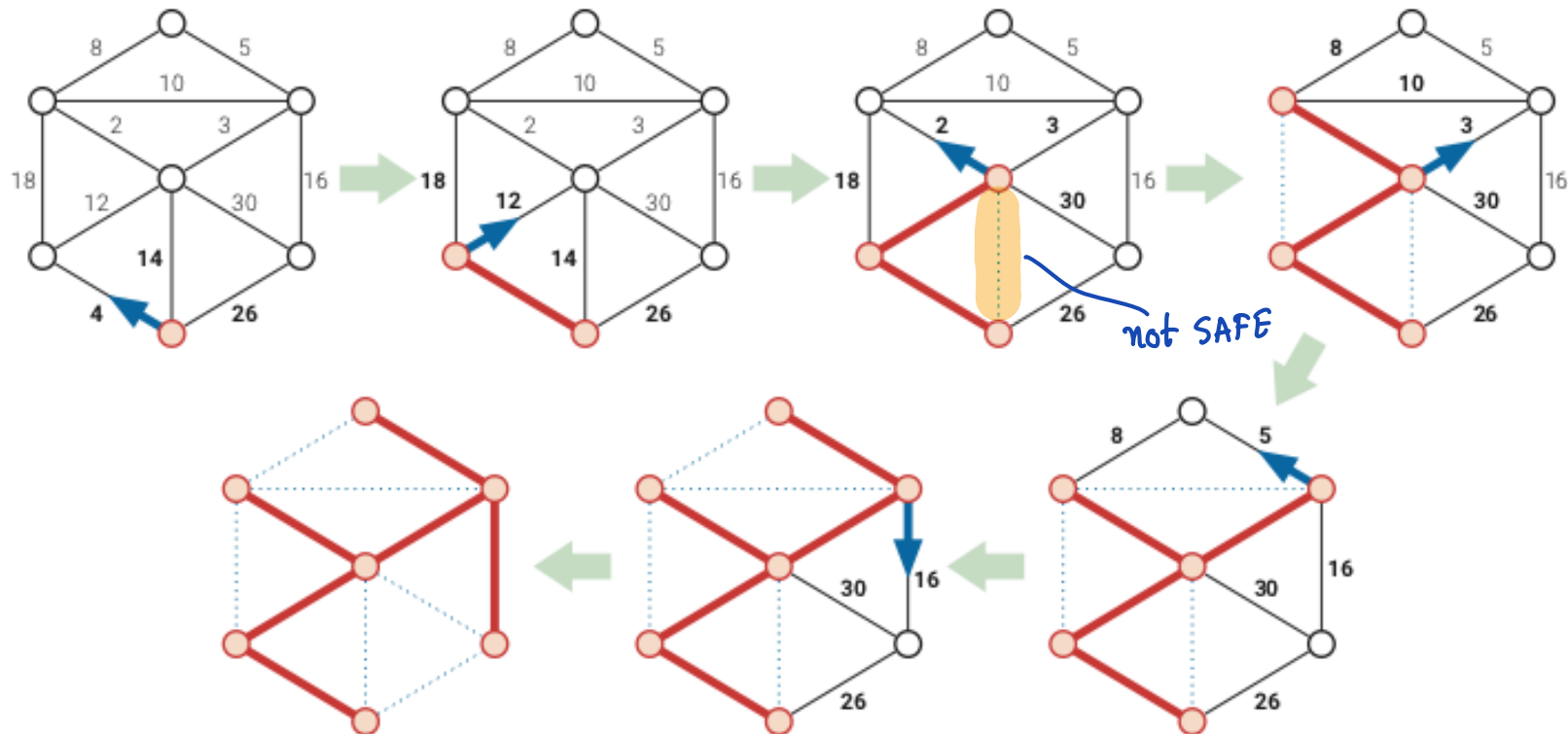
[Jarník:1926] letter to Borůvka [Kruskal:1956] [Prim:1957]
[Dijkstra:1958]

At each step we have one tree component T and (isolated) vertices

Pick the safe edge of min weight attached to T

At each step we have a tree

JARNÍK: Repeatedly add T 's safe edge to T .



Prim-Jarník algorithm

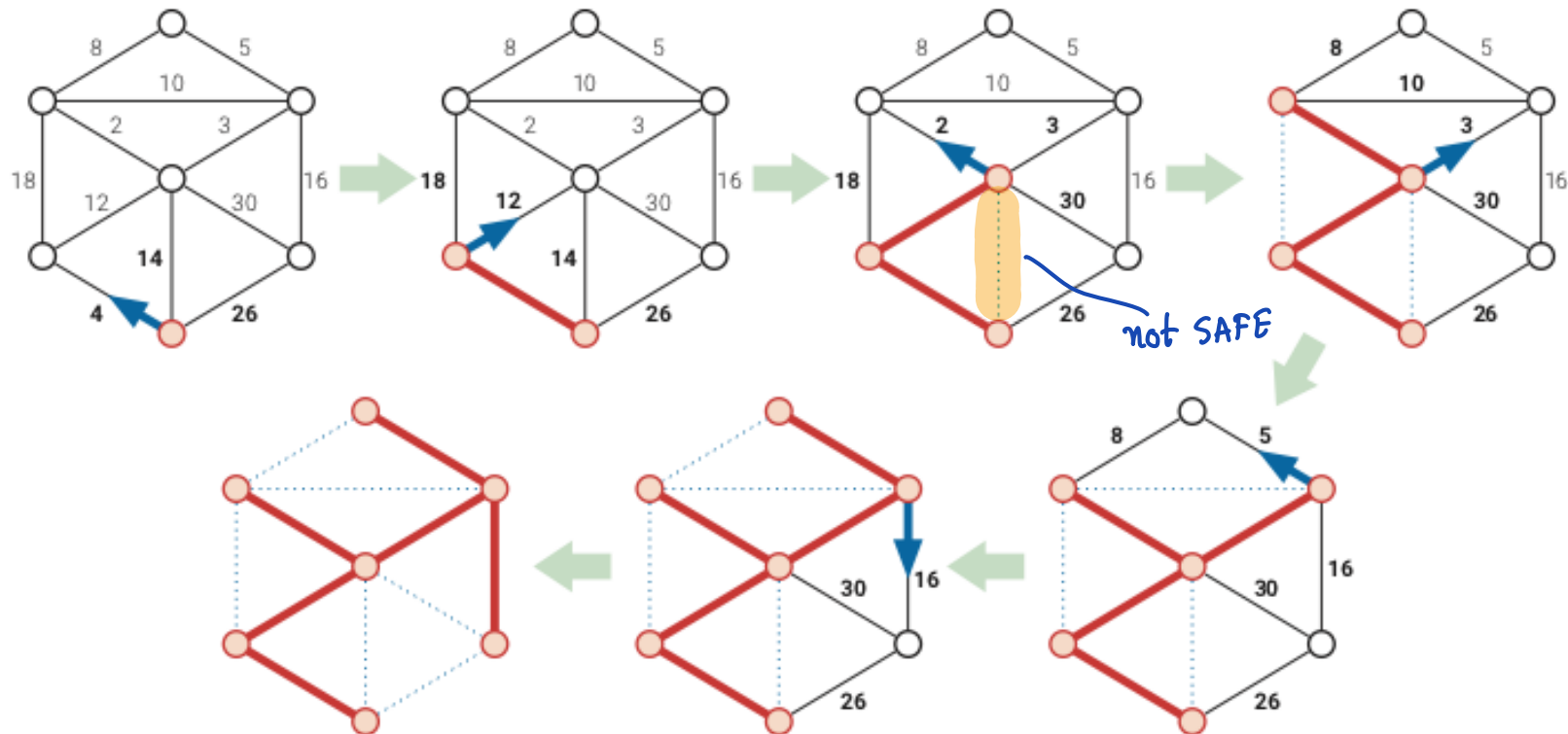
[Jarník:1926] letter to Borůvka [Kruskal:1956] [Prim:1957]
[Dijkstra:1958]

At each step we have one tree component T and (isolated) vertices

Pick the safe edge of min weight attached to T

At each step we have a tree

JARNÍK: Repeatedly add T 's safe edge to T .



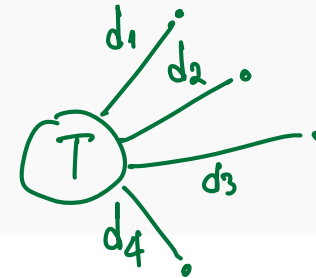
needs a priority queue } $\rightarrow$ heap $\rightarrow$ total complexity $O(E \lg E) = O(E \lg V)$
for the edges

as $E \leq V^2$

Prim-Jarník algorithm + Fibonacci heaps

We can do better : We keep the vertices
For a vertex v , the key is
the min weight distance to
the evolving tree T

there is a
preprocessing



JARNÍK(V, E, s):
JARNÍKINIT(V, E, s)
JARNÍKLOOP(V, E, s)

JARNÍKINIT(V, E, s):
for each vertex $v \in V \setminus \{s\}$
if $vs \in E$
 $edge(v) \leftarrow vs$
 $priority(v) \leftarrow w(vs)$
else
 $edge(v) \leftarrow \text{NULL}$
 $priority(v) \leftarrow \infty$
INSERT(v)

JARNÍKLOOP(V, E, s):
 $T \leftarrow (\{s\}, \emptyset)$
for $i \leftarrow 1$ to $|V| - 1$
 $v \leftarrow \text{EXTRACTMIN}$ FH $O(\lg n)$
 add v and $edge(v)$ to T
 for each neighbor u of v
 if $u \notin T$ and $priority(u) > w(uv)$
 $edge(u) \leftarrow uv$
 DECREASEKEY($u, w(uv)$) → FH $O(1)$ amortized

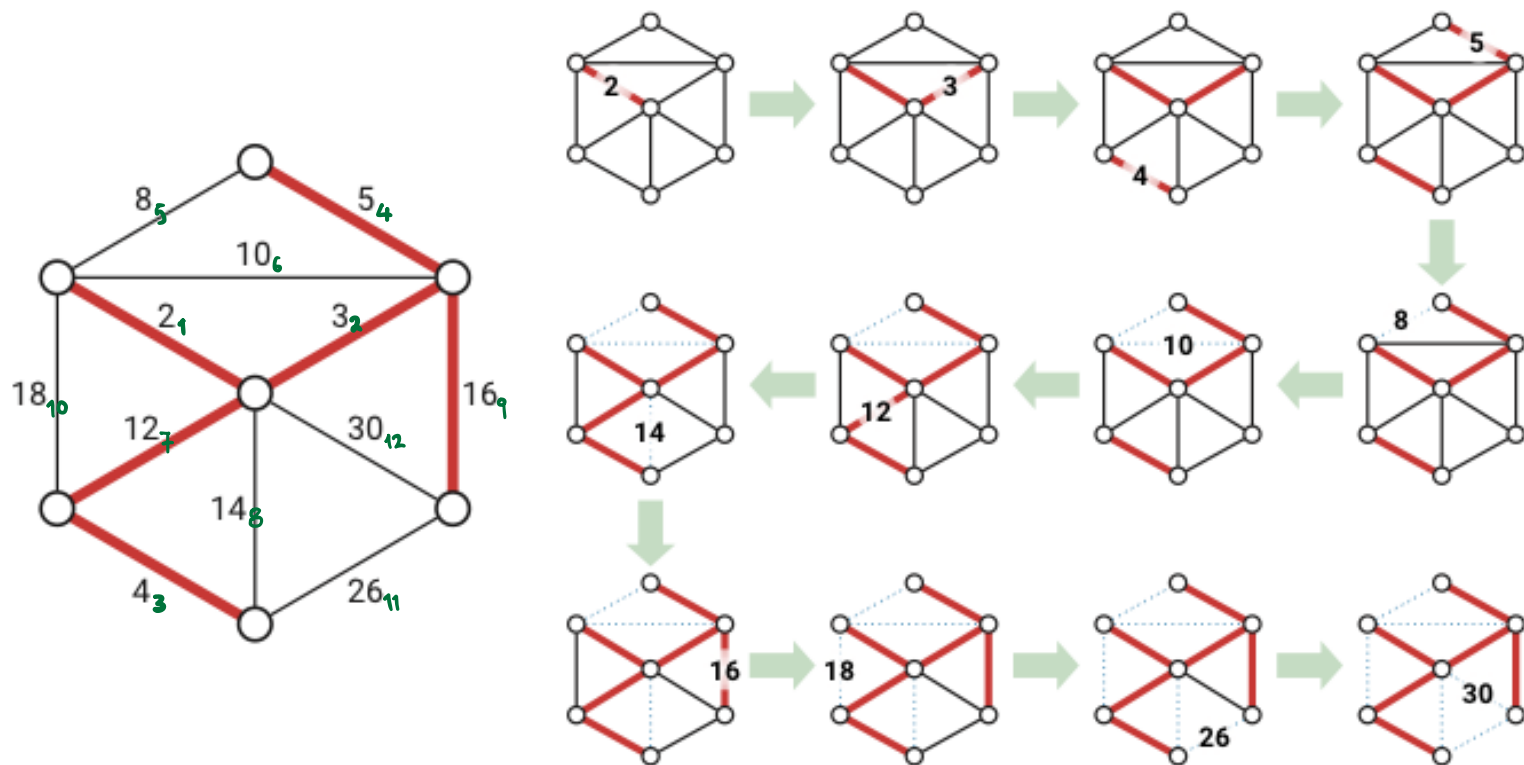
Fibonacci Heap: INSERT
DECREASEKEY } $O(1)$ amortized
EXTRACTMIN $O(\lg n)$

Complexity $O(E + V \lg V)$
Compare with $O(E \lg V)$

Kruskal's algorithm [kruskal : 1956] to simplify Borůvka's algorithm

- sort the edges $O(E \lg E)$
- pick always the min **safe**

KRUSKAL: Scan all edges by increasing weight; if an edge is safe, add it to F .

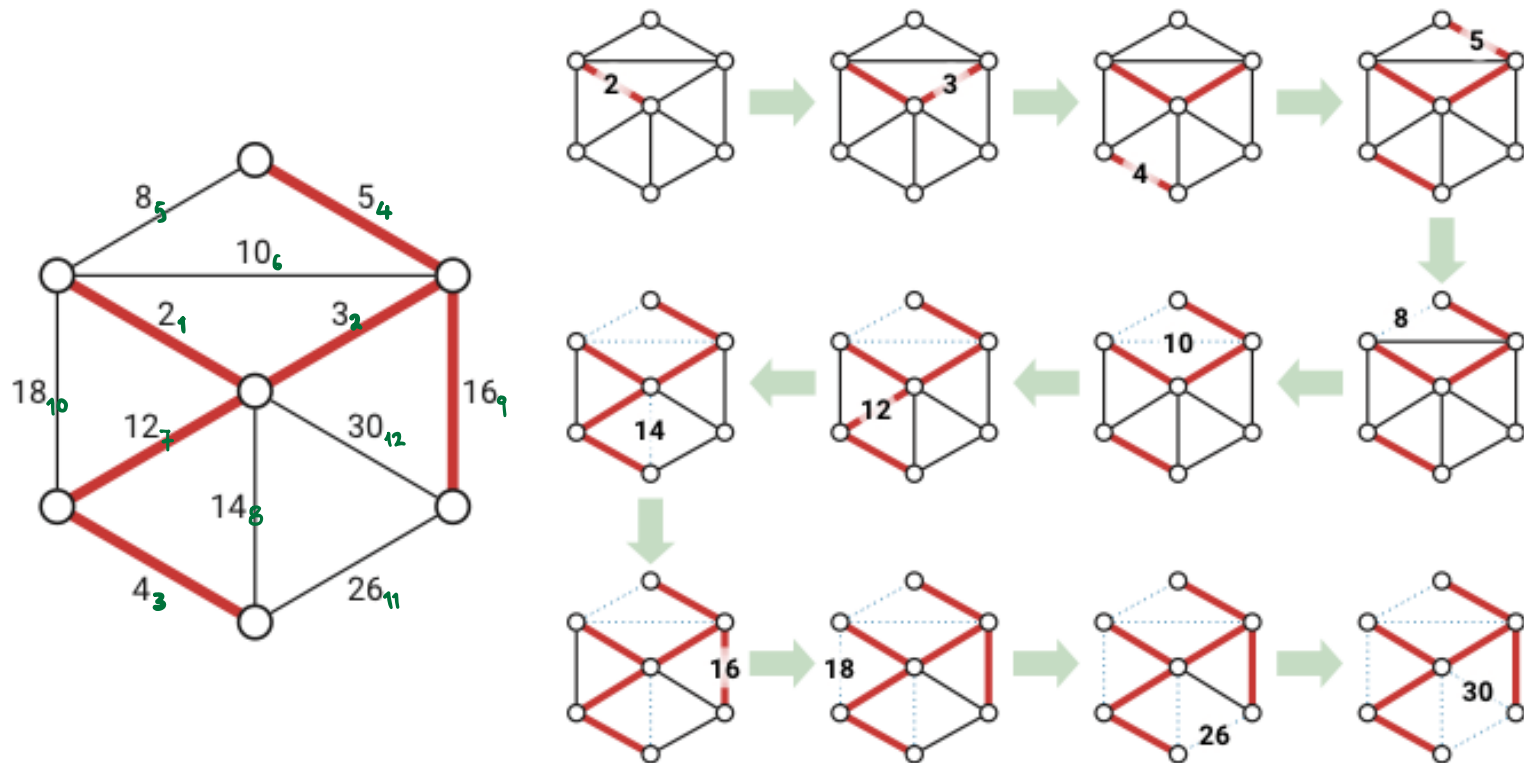


No need to (re-)compute components like Borůvka

Kruskal's algorithm [kruskal : 1956] to simplify Borůvka's algorithm

- sort the edges $O(E \log E)$
- pick always the min **safe**

KRUSKAL: Scan all edges by increasing weight; if an edge is safe, add it to F .



No need to (re-)compute components like Borůvka

- We need to find to which components an edge is attached to
- We need to merge (unify) components

Kruskal and Union-Find

Complexity
runs in $O(E + V \lg V)$
BUT sorting in $O(E \lg E)$

total complexity
 $O(E \lg E) = O(E \lg V)$

KRUSKAL(V, E):

sort E by increasing weight

$F \leftarrow (V, \emptyset)$

for each vertex $v \in V$

 MAKESET(v)

for $i \leftarrow 1$ to $|E|$

$uv \leftarrow$ i th lightest edge in E

 if FIND(u) $\neq$ FIND(v)

 UNION(u, v)

 add uv to F

return F

create a set
containing only v $O(1)$

Find the set
containing v $O(1)$

unite the
sets containing
 u and v $O(\lg V)$
amortized

Contents

MST

Boruvka

Prim

Kruskal

Best MST algorithms

Deterministic $O(E \alpha(V))$ [Chazelle]

↑ inverse Ackermann function

Randomized $O(E)$ [Karger - Klein - Tarjan: 1975]

recall the $\tilde{O}$ notation

Shortest Path(s)

SSSP for unweighted graphs

SSSP for DAGs

Dijkstra

Bellman-Ford

All Pairs Shortest Paths

Contents

MST

Boruvka

Prim

Kruskal

Shortest Path(s)

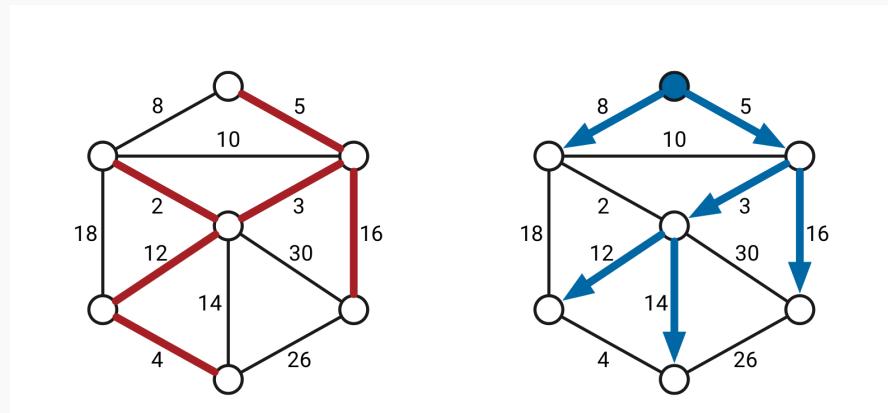
SSSP for unweighted graphs

SSSP for DAGs

Dijkstra

Bellman-Ford

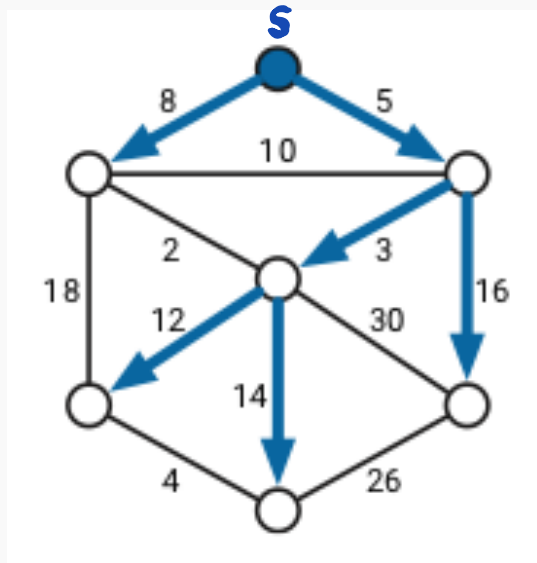
All Pairs Shortest Paths



Single Source Shortest Path (we find the shortest paths from s to all other vertices)

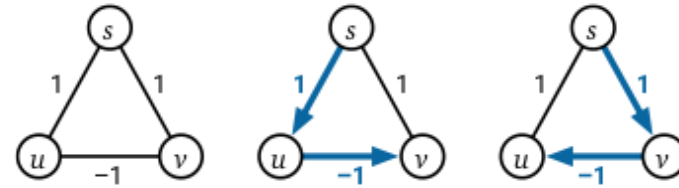
$$G = (V, E, w)$$

they form a tree (why?)
unique? (why?)



P : directed path from s to t .

Negative weights cause a problem : e.g., SSSP is not a tree anymore

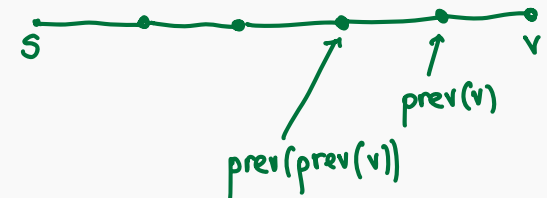


s : source

$\text{dist}(v)$: the length of the tentative shortest path from s to v

$\text{prev}(v)$: the vertex before v in the shortest path from s to v

$$\min_P \{w(P) = \sum_{u \rightarrow v} w(u \rightarrow v)\}$$



Universal algorithm for SSSP [Ford: 1956]

[Dandig: 1957] [Minty: 1958]

Unified

INITSSSP(s):

$dist(s) \leftarrow 0$

$pred(s) \leftarrow \text{NULL}$

for all vertices $v \neq s$

$dist(v) \leftarrow \infty$

$pred(v) \leftarrow \text{NULL}$

RELAX($u \rightarrow v$):

$dist(v) \leftarrow dist(u) + w(u \rightarrow v)$

$pred(v) \leftarrow u$

change/update
the shortest path
it is better to go to v
through u

FORDSSSP(s):

INITSSSP(s)

while there is at least one tense edge

RELAX any tense edge

S: source

$dist(v)$: the length of the tentative
shortest path from s to v

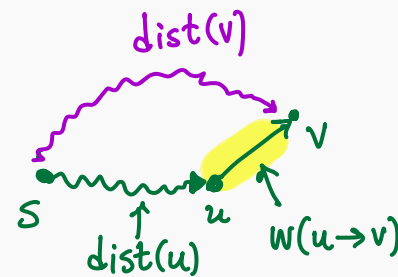
$prev(v)$: the vertex before v in the
shortest path from s to v

TENSE edge ($u \rightarrow v$)

the current shortest path to v
is incorrect.

Thus $s \rightsquigarrow u \rightarrow v$ is shorter

If we go through $u \rightarrow v$
is shorter



$$dist(u) + w(u \rightarrow v) < dist(v)$$

Q: How do we choose
the TENSE edges?

Unweighted graphs (BFS) [Moore: 1957], [Zuse: 1945]

BFS(s):

INITSSSP(s)

PUSH(s)

while the **queue** is not empty

$u \leftarrow \text{PULL}()$

for all edges $u \rightarrow v$

if $\text{dist}(v) > \text{dist}(u) + 1$

«if $u \rightarrow v$ is tense»

update {
 $\text{dist}(v) \leftarrow \text{dist}(u) + 1$
 $\text{pred}(v) \leftarrow u$
 PUSH(v)

«relax $u \rightarrow v$ »

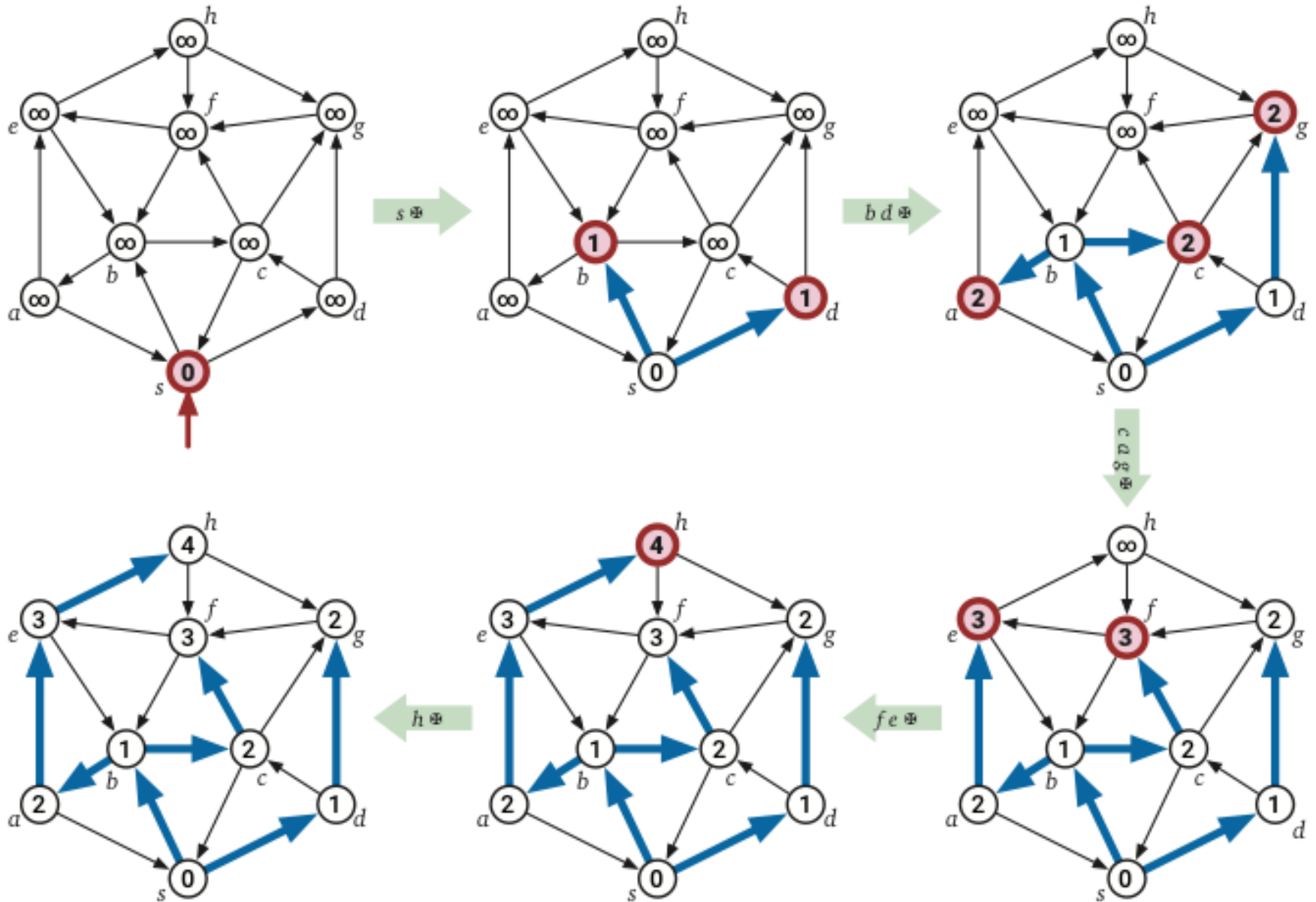
if $\text{dist}(v) = \infty$

Complexity : $O(V+E)$

Unweighted graphs (BFS) :: An example

like a waterfront
or like a moving token

Complexity $O(V+E)$
We visit once every vertex / edge



DAG (DFS)

weighted or not

Can contain negative edges

No negative cycles

How do we do this?

DAGSSSP(s):

for all vertices v in topological order

if $v = s$

$dist(v) \leftarrow 0$

else

$dist(v) \leftarrow \infty$

for all edges $u \rightarrow v$

update

$\left\{ \begin{array}{l} \text{if } dist(v) > dist(u) + w(u \rightarrow v) \\ dist(v) \leftarrow dist(u) + w(u \rightarrow v) \end{array} \right.$

$\langle\langle \text{if } u \rightarrow v \text{ is tense} \rangle\rangle$
 $\langle\langle \text{relax } u \rightarrow v \rangle\rangle$

consider the incoming edges to v
and update if necessary

Complexity
 $O(V+E)$

formulation à la Ford

$$dist(v) = \begin{cases} 0 & \text{if } v = s \\ \min_{u \rightarrow v} (dist(u) + w(u \rightarrow v)) & \text{otherwise} \end{cases}$$

DP

DAGSSSP(s):

INITSSSP(s)

for all vertices v in topological order

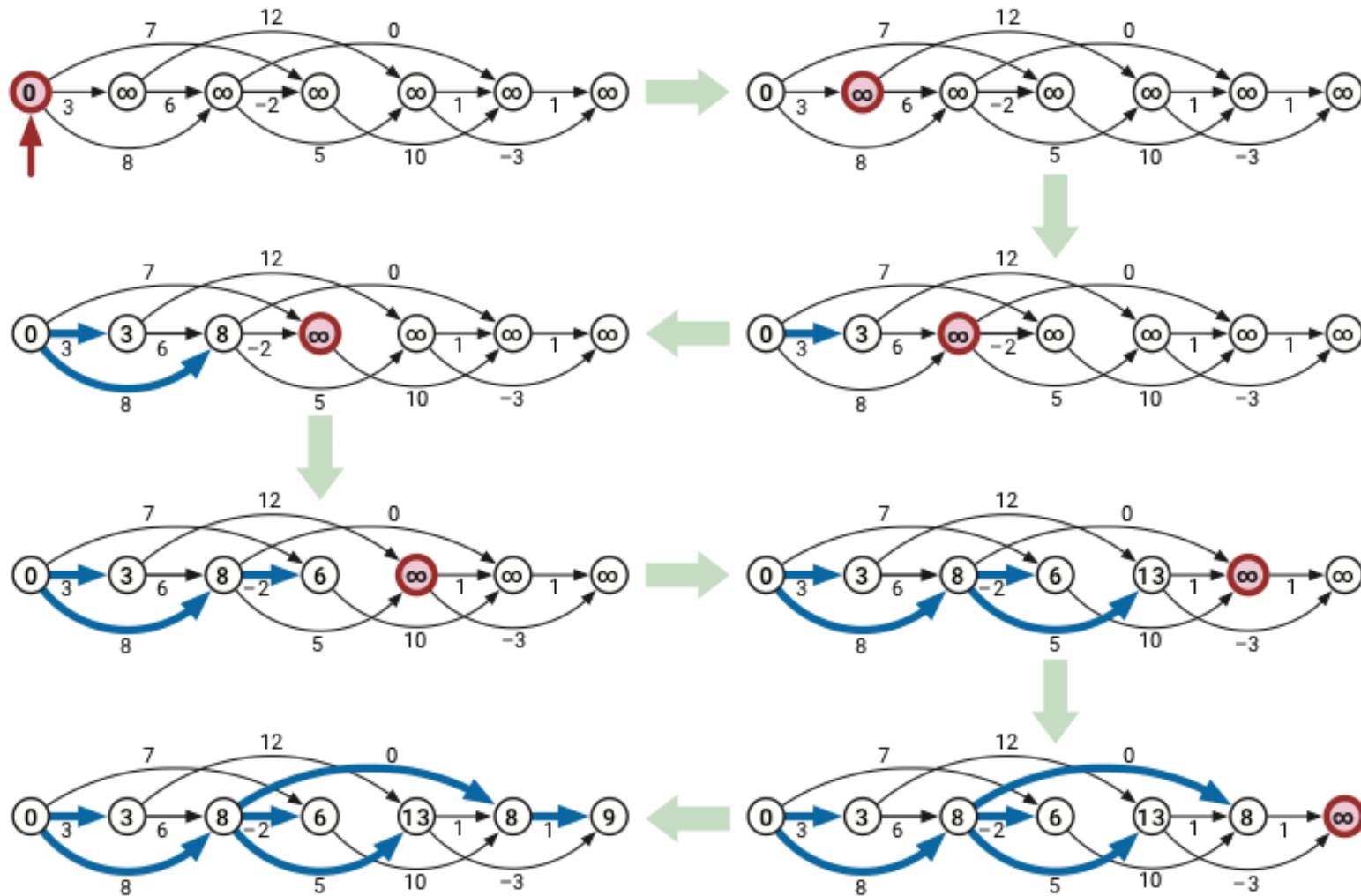
for all edges $u \rightarrow v$

if $u \rightarrow v$ is tense

RELAX($u \rightarrow v$)

DAG (DFS) :: an example

Consider a topological order : For each v (in order) consider the incoming edges



Best First: Dijkstra [Dijkstra : 1957-1959] and many others

No negative weight edges

Consider the vertex with the minimum distance so far and update its neighbours

```
DIJKSTRA(s):  
  INITSSSP(s)  
  INSERT(s, 0)  
  while the priority queue is not empty  
     $u \leftarrow \text{EXTRACTMIN}()$   
    for all edges  $u \rightarrow v$   
      if  $u \rightarrow v$  is tense  
        RELAX( $u \rightarrow v$ )  
      if  $v$  is in the priority queue  
        DECREASEKEY( $v, \text{dist}(v)$ )  
      else  
        INSERT( $v, \text{dist}(v)$ )
```

the key for every vertex is the tentative $\text{dist}(v)$

Each vertex is visited once but its tentative distance, $\text{dist}(v)$ can change many times

Complexity: $O(E \lg V)$

to find the min weight edge
DECREASE operations

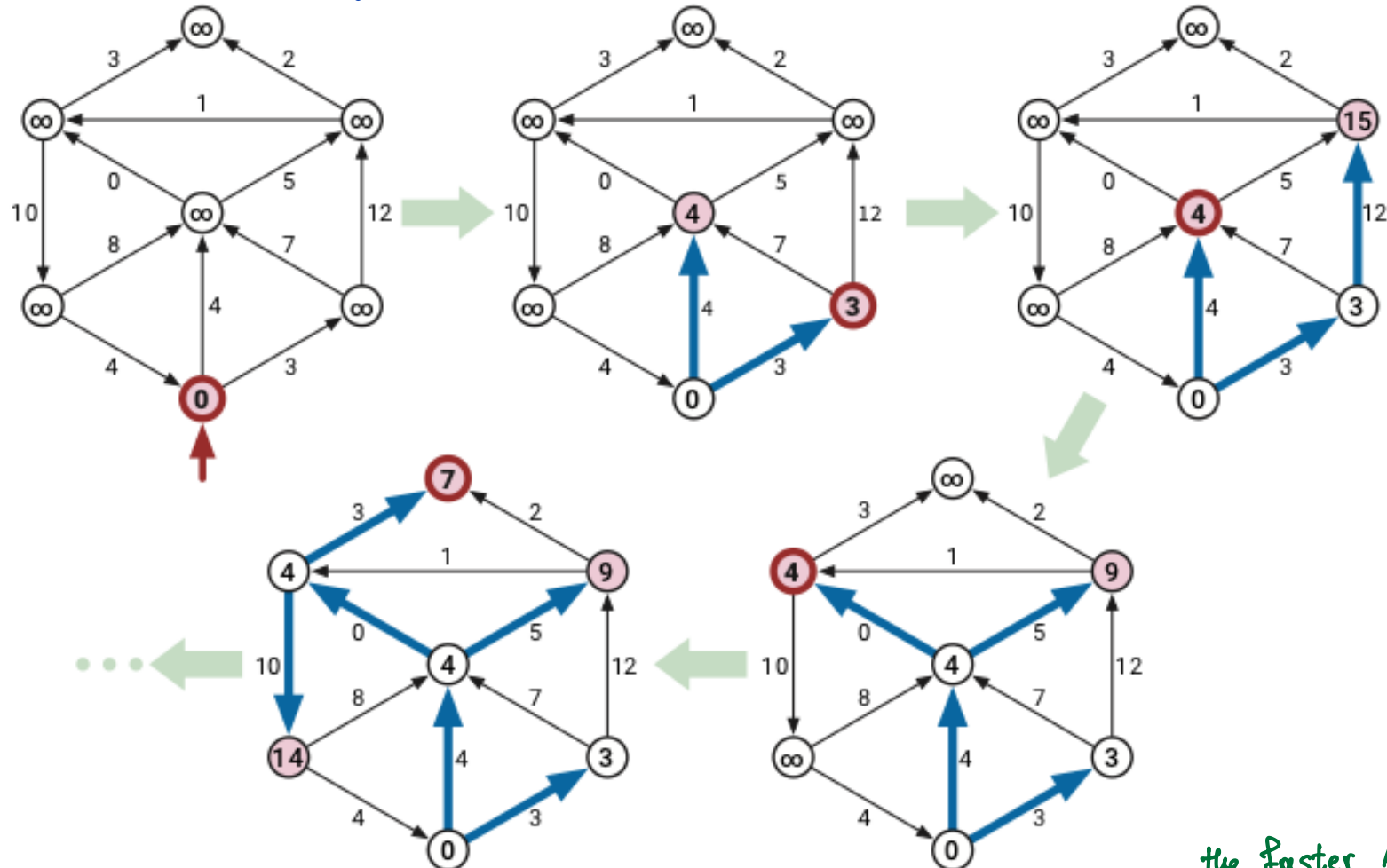
Dijkstra :: an example

initially the queue contains S

Pop(s)

Consider the neighbours : if they are tense update and put in the queue

Consider the vertex with the minimum distance so far and update its neighbours



the faster ALG
in practice

Relax all the edges (Bellman-Ford)

[Shimbel: 1954] [Moore: 1957] [Bellman: 1958]
and many others

It allows negative ^{edges} _{cycles}

INITSSSP(s):

$dist(s) \leftarrow 0$

$pred(s) \leftarrow \text{NULL}$

for all vertices $v \neq s$

$dist(v) \leftarrow \infty$

$pred(v) \leftarrow \text{NULL}$

RELAX($u \rightarrow v$):

$dist(v) \leftarrow dist(u) + w(u \rightarrow v)$

$pred(v) \leftarrow u$

BELLMANFORD(s)

INITSSSP(s)

while there is at least one tense edge

for every edge $u \rightarrow v$

if $u \rightarrow v$ is tense

RELAX($u \rightarrow v$)

Relax all the edges :: an example

BELLMANFORD(s)

INITSSSP(s)

repeat $V - 1$ times

for every edge $u \rightarrow v$

if $u \rightarrow v$ is tense

RELAX($u \rightarrow v$)

for every edge $u \rightarrow v$

if $u \rightarrow v$ is tense

return "Negative cycle!"

BELLMANFORDFINAL(s)

$dist[s] \leftarrow 0$

for every vertex $v \neq s$

$dist[v] \leftarrow \infty$

for $i \leftarrow 1$ to $V - 1$

for every edge $u \rightarrow v$

if $dist[v] > dist[u] + w(u \rightarrow v)$

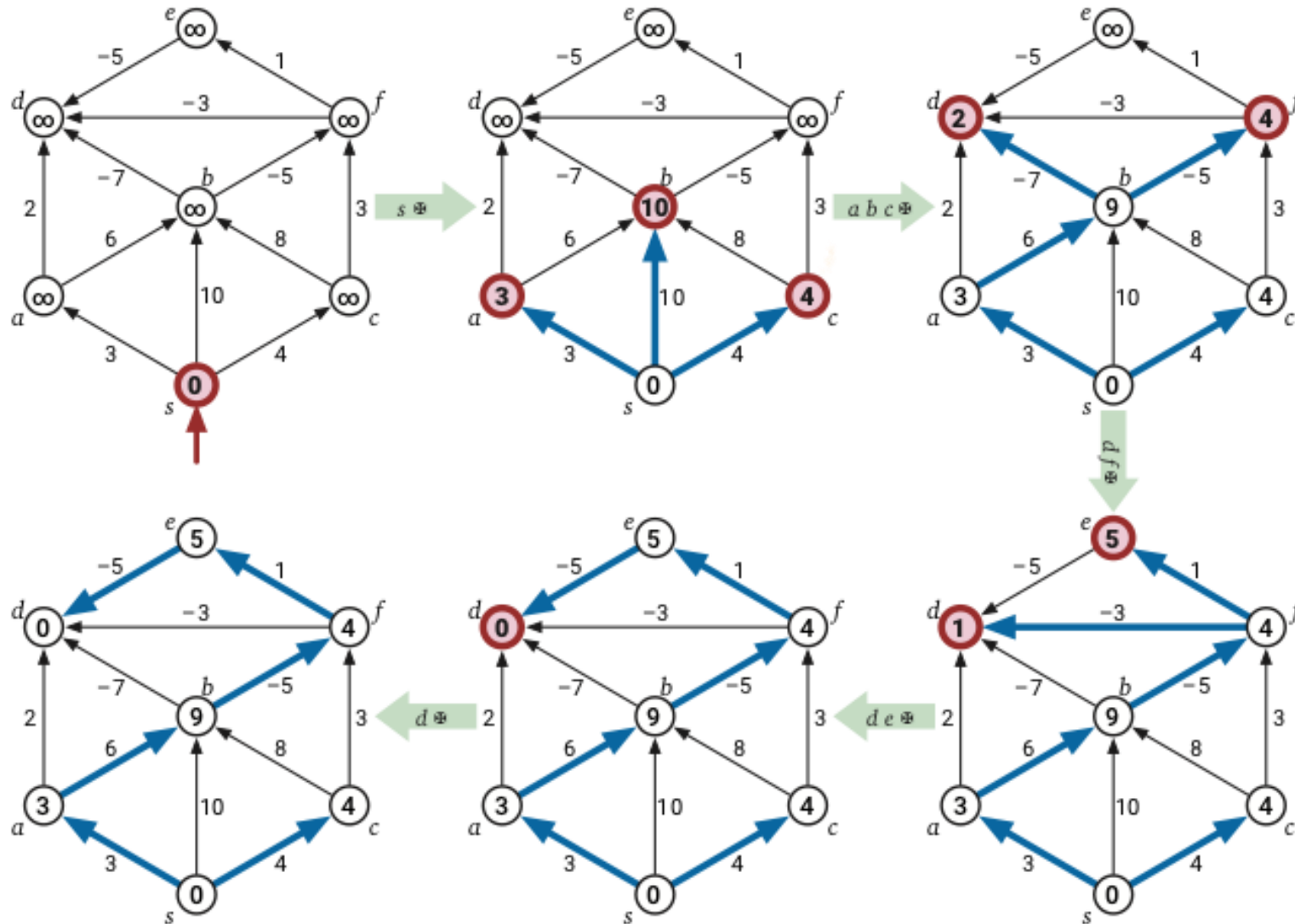
$dist[v] \leftarrow dist[u] + w(u \rightarrow v)$

DP

Complexity : $O(V E)$

Bellman-Ford (à la Moore) :: an example

uses a queue and checks
not to insert a vertex
that it is already in the queue



$O(V E)$
 $\downarrow$
 $O(V^3)$

NOTE: The distance of the shortest path for each vertex is updated many times

Contents

MST

Boruvka

Prim

Kruskal

Shortest Path(s)

SSSP for unweighted graphs

SSSP for DAGs

Dijkstra

Bellman-Ford

All Pairs Shortest Paths

The obvious algorithm

iterations
"V"

OBVIOUSAPSP(V, E, w):

for every vertex s

$dist[s, \cdot] \leftarrow \text{SSSP}(V, E, w, s)$

Run SSSP # V times
i.e. from every vertex

Complexity :

- unweighted $O(VE) = O(V^3)$
- DAG $O(VE) = O(V^3)$
- Dijkstra $O(VE \lg V) = O(V^3 \lg V)$
- BF $O(V^2E) = O(V^4)$

The obvious algorithm

Run V times SSSP

iterations
 V

OBVIOUSAPSP(V, E, w):

for every vertex s

$\text{dist}[s, \cdot] \leftarrow \text{SSSP}(V, E, w, s)$

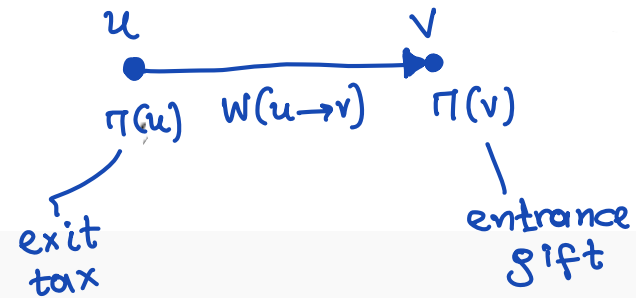
Complexity :

- unweighted $O(VE) = O(V^3)$
- DAG $O(VE) = O(V^3)$
- Dijkstra $O(VE \lg V) = O(V^3 \lg V)$
- BF $O(V^2E) = O(V^4)$

Can we do something better than $O(V^4)$
when there are negative weights?

Reweighting:

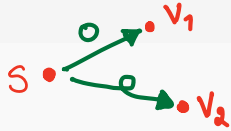
assign weights to the vertices



$$w'(u \rightarrow v) = \pi(u) + w(u \rightarrow v) - \pi(v)$$

In this way the shortest paths
do not change why?

Johnson's approach



if this is not
the case then
add dummy edges
from s to these vertices

JOHNSONAPSP(V, E, w) :

⟨⟨Add an artificial source⟩⟩

add a new vertex s

for every vertex v

add a new edge $s \rightarrow v$

$w(s \rightarrow v) \leftarrow 0$

⟨⟨Compute vertex prices⟩⟩

$\text{dist}[s, \cdot] \leftarrow \text{BELLMANFORD}(V, E, w, s)$

if BELLMANFORD found a negative cycle

fail gracefully

⟨⟨Reweight the edges⟩⟩

for every edge $u \rightarrow v \in E$

$w'(u \rightarrow v) \leftarrow \text{dist}[s, u] + w(u \rightarrow v) - \text{dist}[s, v]$

⟨⟨Compute reweighted shortest path distances⟩⟩

for every vertex u

$\text{dist}'[u, \cdot] \leftarrow \text{DIJKSTRA}(V, E, w', u)$

⟨⟨Compute original shortest-path distances⟩⟩

for every vertex u

for every vertex v

$\text{dist}[u, v] \leftarrow \text{dist}'[u, v] - \text{dist}[s, u] + \text{dist}[s, v]$

• Assume there is a vertex s
from which we can reach any other vertex

• Run BF from s
and for each v
set $\pi(v) = \text{dist}(s, v)$

now

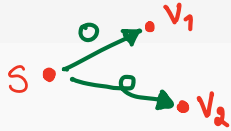
$$w'(u \rightarrow v) = \underbrace{\text{dist}(s, u)}_{\pi(u)} + w(u \rightarrow v) - \underbrace{\text{dist}(s, v)}_{\pi(v)}$$

• Run Dijkstra V times

Complexity $O(V^3 \lg V)$

even with negative edges
(BUT no negative cycles)

Johnson's approach



if this is not
the case then
add dummy edges
from s to these vertices

JOHNSONAPSP(V, E, w) :

⟨⟨Add an artificial source⟩⟩

add a new vertex s

for every vertex v

add a new edge $s \rightarrow v$

$w(s \rightarrow v) \leftarrow 0$

⟨⟨Compute vertex prices⟩⟩

$\text{dist}[s, \cdot] \leftarrow \text{BELLMANFORD}(V, E, w, s)$

if BELLMANFORD found a negative cycle

fail gracefully

⟨⟨Reweight the edges⟩⟩

for every edge $u \rightarrow v \in E$

$w'(u \rightarrow v) \leftarrow \text{dist}[s, u] + w(u \rightarrow v) - \text{dist}[s, v]$

⟨⟨Compute reweighted shortest path distances⟩⟩

for every vertex u

$\text{dist}'[u, \cdot] \leftarrow \text{DIJKSTRA}(V, E, w', u)$

⟨⟨Compute original shortest-path distances⟩⟩

for every vertex u

for every vertex v

$\text{dist}[u, v] \leftarrow \text{dist}'[u, v] - \text{dist}[s, u] + \text{dist}[s, v]$

• Assume there is a vertex s
from which we can reach any other vertex

• Run BF from s

and for each v

set $\pi(v) = \text{dist}(s, v)$

now

$$w'(u \rightarrow v) = \underbrace{\text{dist}(s, u)}_{\pi(u)} + w(u \rightarrow v) - \underbrace{\text{dist}(s, v)}_{\pi(v)}$$

• Run Dijkstra V times

Complexity $O(V^3 \lg V)$

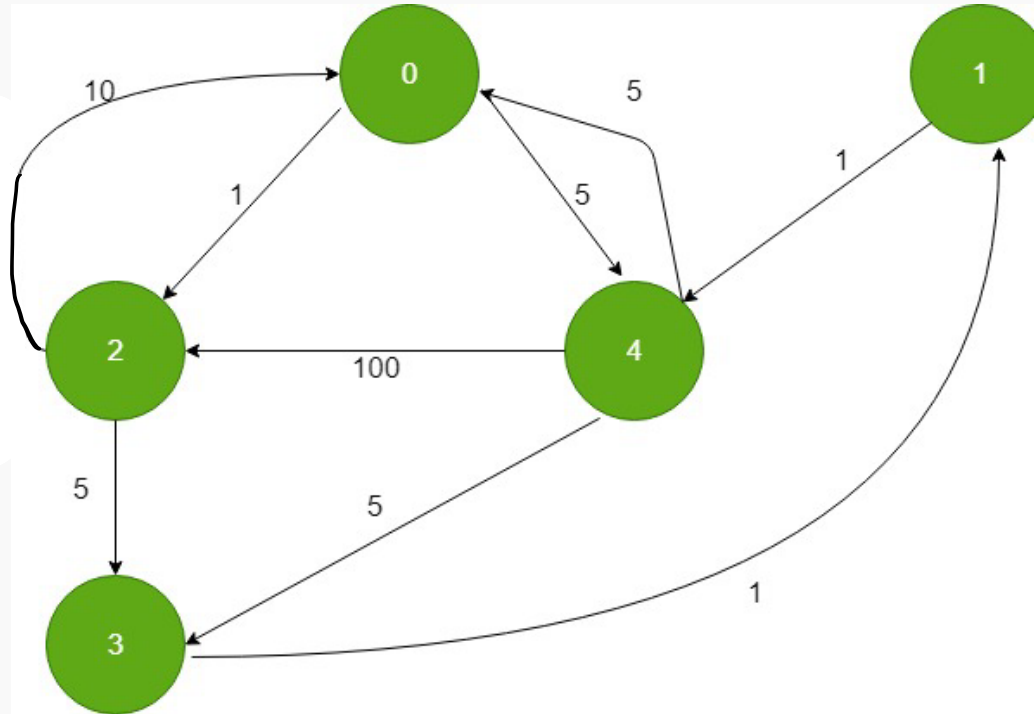
even with negative edges
(BUT no negative cycles)

[Floyd-Warshall] Algorithm DP $O(V^3)$

$$\text{dist}(u, v, r) = \begin{cases} w(u \rightarrow v) & r=0 \\ \min \left\{ \begin{array}{l} \text{dist}(u, v, r-1) \\ \text{dist}(u, r, r-1) + \text{dist}(r, v, r-1) \end{array} \right\} & r>0 \end{cases}$$

SPACE $O(V^3)$

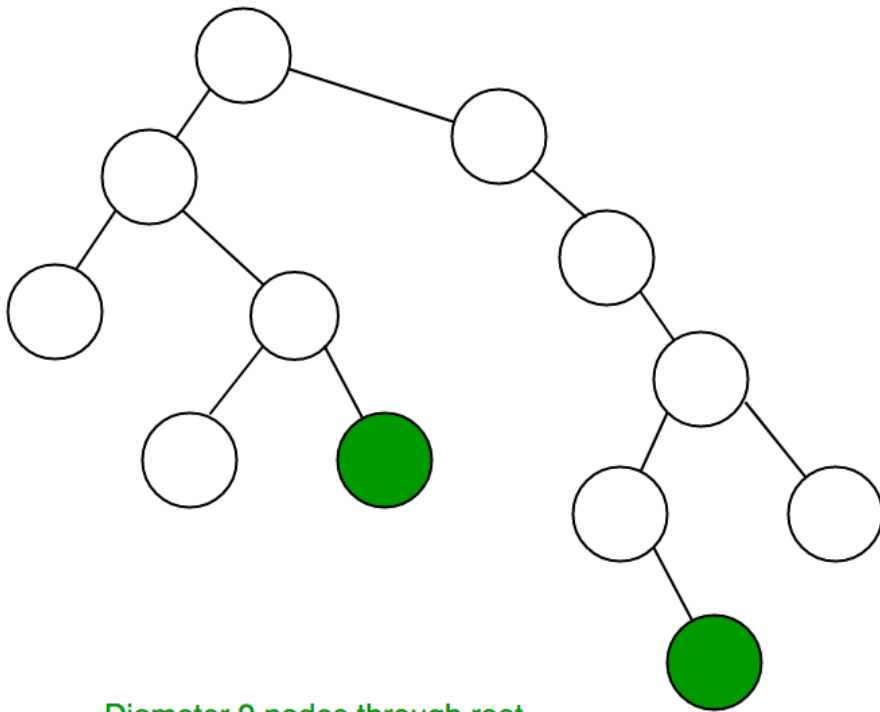
An interesting question 1/5



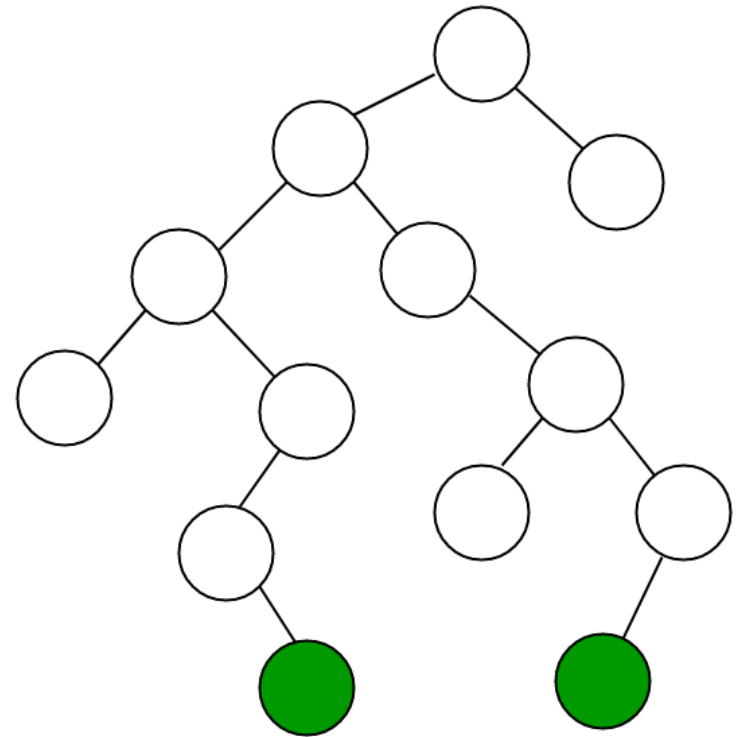
Find the shortest path to a (target) vertex
from all other vertices

An interesting question 2/5

Diameter of a tree

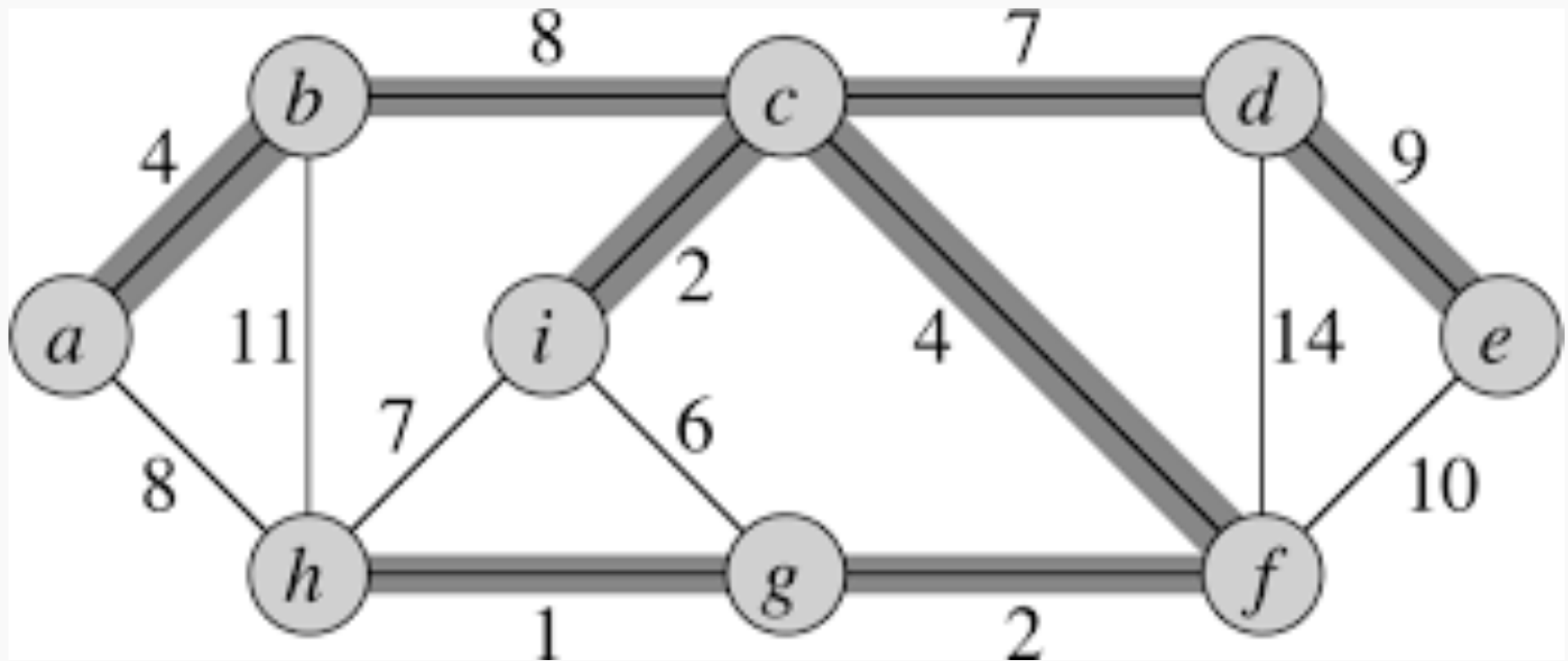


Diameter 9 nodes through root



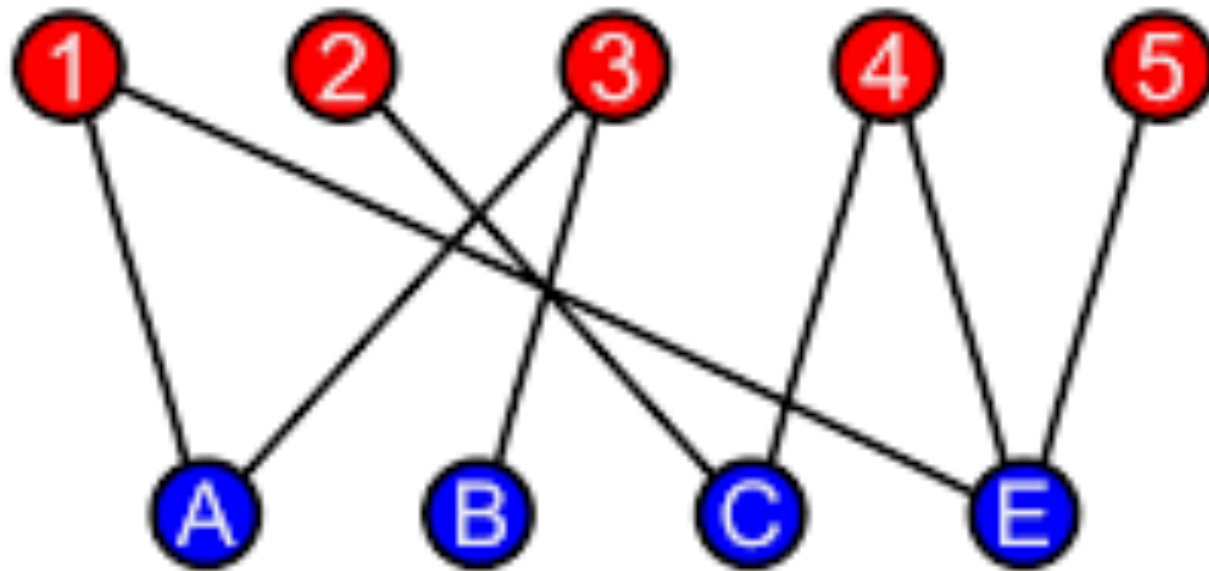
Diameter 9 nodes not through root

An interesting question 3/5



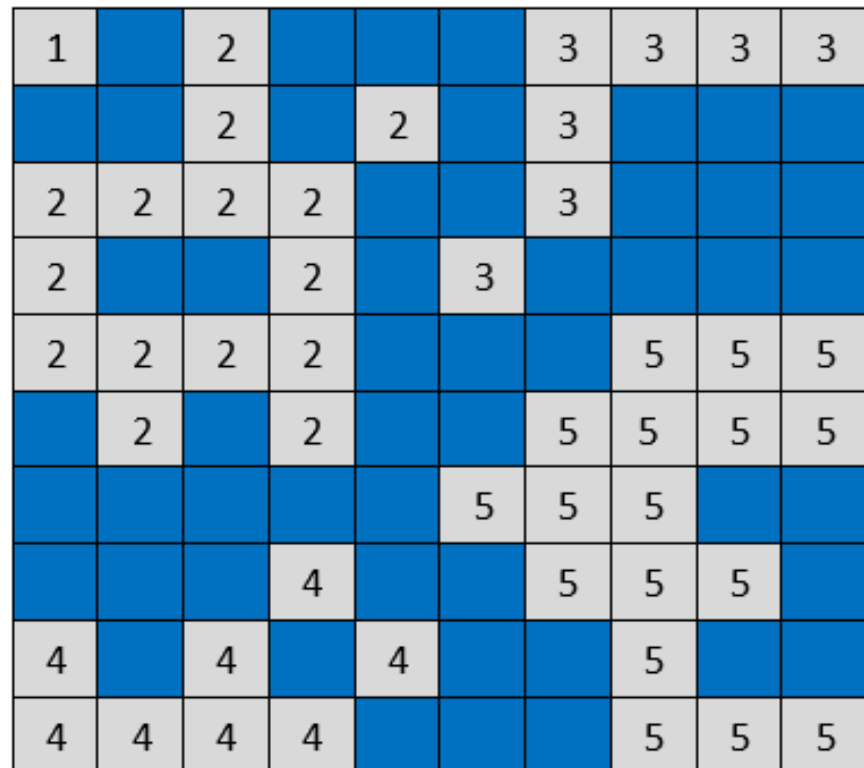
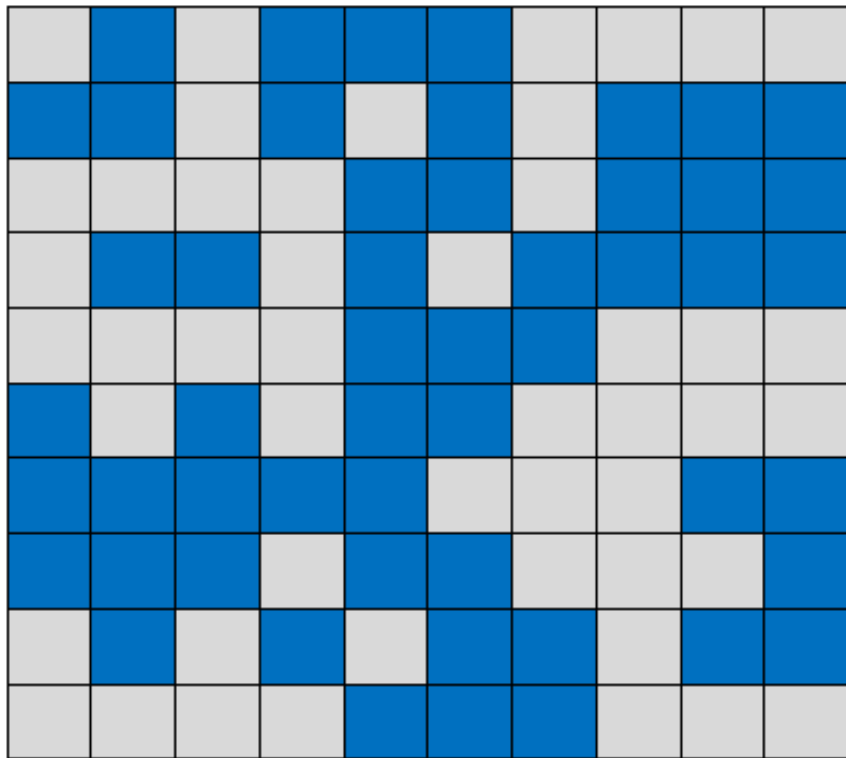
Minimum **product** Spanning Tree

An interesting question 4/5



(Recognizing) Bipartite graphs

An interesting question 5/5



Count the number of islands